

Multi-View Human Tracking with Puzzleboard Calibration

Overview

You are given videos from **4 cameras** observing the same room. In the scene, an actor performs calibration using a **Puzzleboard** target. For this assignment, you may treat the **Puzzleboard as a chessboard-like planar calibration object**. Each puzzle piece has size **7 cm**.

The main goal is to build a simple **multi-view human tracking pipeline** that can:

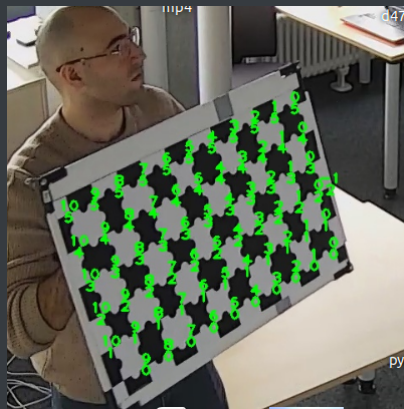
- detect human keypoints in each camera,
- use geometry between cameras,
- recover missing keypoints from other views,
- and combine detections into a more stable result.

A practical challenge is that the cameras may not be perfectly synchronized, so small **temporal offsets** may exist.

Github repo for detector:

<https://github.com/PStelldinger/PuzzleBoard>

Part A — Camera Calibration and 2D Keypoints



Task A1 — Monocular Calibration

Using the Puzzleboard repository, calibrate each camera **individually**.

For each camera, estimate:

- intrinsic matrix K
- distortion coefficients D

Requirements

- Use at least **50 calibration frames per camera**
- Make sure the Puzzleboard appears in different image regions and orientations

Deliverables

- K and D for each camera
 - a few example calibration frames
-

Task A2 — 2D Human Keypoint Detection

Run a 2D pose detector on the actor for each camera stream.

Possible options:

- YOLO pose
- MMPose
- or another equivalent method

For each frame and camera, store:

- detected keypoint coordinates
- confidence values
- missing / low-confidence joints

Deliverables

- the chosen detector
 - example frames with keypoints drawn
-

Task A3 — Temporal Offset Estimation

Estimate rough temporal offsets between the cameras.

Example: frame $t=0$ in one camera may correspond to frame $t=8$ in another.

Use visible events such as:

- foot contact with the floor
- an object touching a table
- sudden motion changes

Deliverables

- estimated offsets between camera pairs
-

Part B — Best-View Rendering

Task B1 — Best Camera Selection

Implement a visualization that shows all 4 camera views with detected keypoints.

At each time step, score each camera based on:

- number of detected keypoints (Most important)
- average confidence (Second next)

Then select the current **best view** and display it in an extra panel.

Deliverables

- the visualization (Also please write the values as text on the frames itself.)
-

Part C — Fundamental Matrix Estimation

Use synchronized frames and Puzzleboard correspondences to estimate the **fundamental matrix** between selected camera pairs.

Task C1 — Fundamental Matrix with OpenCV

Estimate the fundamental matrix using OpenCV.

Task C2 — Fundamental Matrix from Essential Matrix with OpenCV

Normalize the pixels (Using our known intrinsics), and estimate the Essential matrix. Afterwards, using the known intrinsics construct the Fundamental matrix from the Essential Matrix.

Task C3 — Fundamental Matrix with Your Own Implementation

Estimate the fundamental matrix again using **your own manual matrix-based implementation**.

This is the only part that must be implemented manually.

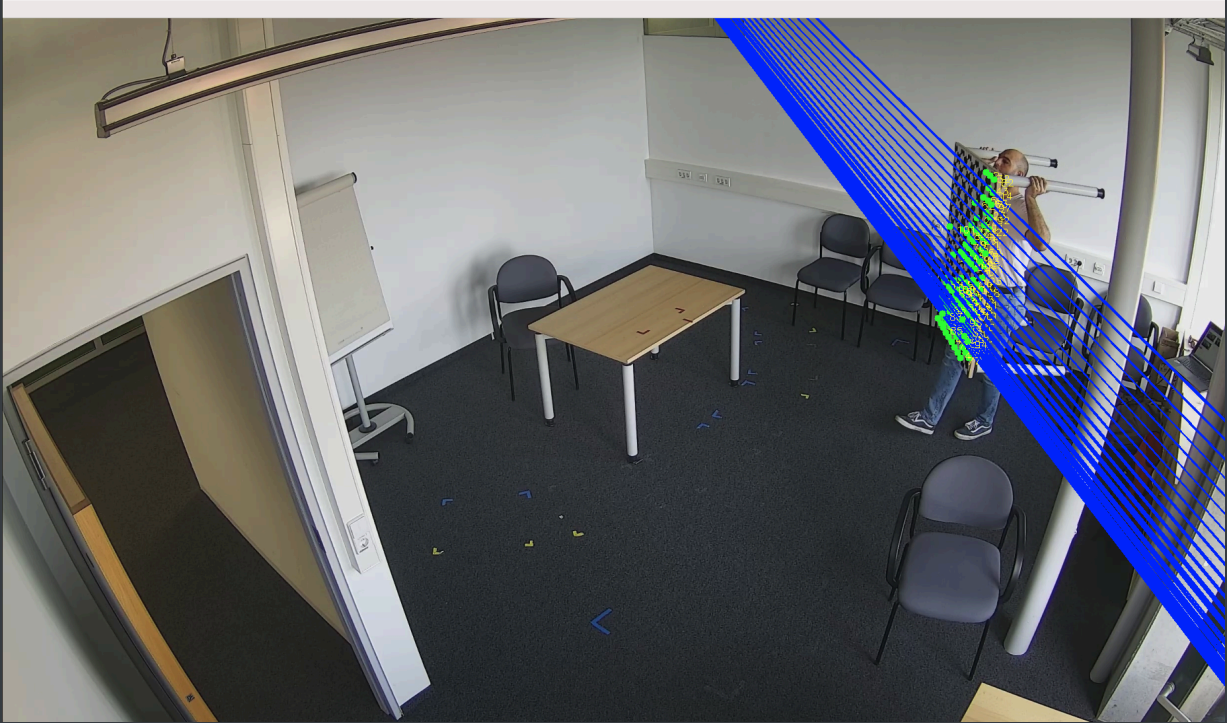
Deliverables

- OpenCV results
 - custom implementation result
 - a short explanation of your custom method
-

Part D — Visual Validation with Epipolar Lines

Instead of using numerical error metrics for now, validate the result **visually**.

Task D1 — Draw Epipolar Lines



Take a point from one camera and project its epipolar line into the other camera using the estimated fundamental matrix.

Then check whether the corresponding keypoint in the other image lies close to that line.

This should be done for:

- Puzzleboard points
- actor keypoints where possible

Important

You should **visualize these epipolar lines for debugging**.

This is one of the main validation tools in the project.

If the lines do not match the corresponding points well, possible issues include:

- wrong synchronization
- bad point correspondences
- poor calibration
- incorrect matrix estimation

Deliverables

- several images showing projected epipolar lines
 - a short discussion of whether they match the expected keypoints (And which method worked the best)
-

Part E — Missing Keypoint Recovery

Core Idea

Imagine that you track **5 keypoints** on the actor and you have **3 cameras**:

- camera 0
- camera 1
- camera 2

At one frame, camera 0 detects only **4 of the 5 keypoints**. One keypoint is missing.

Luckily, camera 1 and camera 2 both detect that missing keypoint.

Now the idea is:

- the detected keypoint in camera 1 creates an **epipolar line** in camera 0
- the detected keypoint in camera 2 creates another **epipolar line** in camera 0

The missing keypoint in camera 0 should lie on both lines.

So ideally, the intersection of the two lines gives the missing keypoint location.

Task E1 — Epipolar-Line-Based Recovery

Implement a simple missing-keypoint recovery method:

1. detect that a keypoint is missing in camera 0
2. take the same keypoint from camera 1 and camera 2

3. compute the two epipolar lines in camera 0
4. visualize the lines
5. estimate the missing point from their intersection

Important

You should **always draw the two lines and the estimated intersection point** for debugging.

If the lines do not intersect perfectly

In real data, the lines may not meet exactly because of:

- noisy detections
- synchronization errors
- imperfect geometry

In such cases, suggest or implement a simple fallback, for example:

- use the point that minimizes distance to both lines
- reject cases where the lines are nearly parallel

Deliverables

- the recovery method
- visual examples
- a short explanation of what happens when the lines do not intersect cleanly

Submission Checklist

Your submission should include:

- calibration results (K, D) for each camera
- estimated temporal offsets
- keypoint detection examples
- best-view visualization

- fundamental matrices from:
 - OpenCV
 - OpenCV - Essential Computation -> Fundamental Construction Approach
 - your own implementation
 - visual epipolar-line validation
 - missing-keypoint recovery examples
-

Suggested Report Structure

1. Introduction
2. Data Setup
3. Monocular Calibration
4. Temporal Synchronization
5. 2D Keypoint Detection
6. Best-View Rendering
7. Fundamental Matrix Estimation
8. Epipolar-Line Visualization and Validation
9. Missing Keypoint Recovery
10. Discussion and Limitations
11. Conclusion